arXiv:2411.16341v1 [cs.PL] 25 Nov 2024

FROM CISC TO RISC: LANGUAGE-MODEL GUIDED ASSEMBLY TRANSPILATION

Ahmed Heakl * **Chaimaa Abi** * **Rania Hossam** **Abdulrahman Mahmoud**

Mohamed bin Zayed University of Artificial Intelligence, Abu Dhabi, UAE

ABSTRACT

The transition from x86 to ARM architecture is becoming increasingly common across various domains, primarily driven by ARM's energy efficiency and improved performance across traditional sectors. However, this ISA shift poses significant challenges, mainly due to the extensive legacy ecosystem of x86 software, and lack of portability across proprietary ecosystems and software stacks. This paper introduces CRT, a lightweight LLM-based transpiler that automatically converts x86 assembly to ARM assembly. Our approach bridges the fundamental architectural gap between x86's CISC-based and ARM's RISC-based computing paradigms while preserving program semantics and optimizing performance. We evaluate CRT on diverse real-world applications, achieving 79.25% translation accuracy from x86 to ARMv5 on our comprehensive test suite, and a 88.68% accuracy from x86 to RISC-V. In practical deployments on Apple M2 hardware (ARMv8), our transpiled code achieves 1.73x speedup compared to Apple's Rosetta 2 virtualization engine, while delivering 2.41x memory efficiency and 1.47x better energy consumption. Through testing and analysis, we show that CRT successfully navigates the CISC/RISC divide, and generates correctly executable RISC code despite machine "language" barriers. We release our code, models, training datasets, and benchmarks at: https://ahmedheakl.github.io/asm2asm/

1 INTRODUCTION

The ending of Moore's Law and Dennard scaling has led to a paradigm shift in the way modern processors are designed and architected. No longer benefiting from generational improvement in power, performance, and area efficiency (PPA), many academic and industry players are rethinking their architectural designs. This includes both introducing more specialized hardware components (such as tensor cores for ML processing) (Markidis et al., 2018; Jouppi et al., 2017), alternatives to Von Neumann architectures (e.g., processing-in-memory (Han et al., 2020; Kang et al., 2013), and revisting the fundemental computing paradigm of CISC versus RISC designs.

Complex instruction set computers (or CISC) such as Intel and AMD's x86 instruction set architecture (ISA) have maintained a stronghold in the datacenter and server space (holding more than 79.9% of the market (Grand View Research, 2024) as well as personal computing devices (with more than 82% market share in 2024 (Rahman et al., 2024)). Reduced instruction set computers (RISC), on the other hand, are predominantly deployed in energy-constrained environments, such as IoT devices, mobile phones, and edge devices (Grand View Research, 2024). In this space, ARM corporation is the predominant player, licensing its architecture to many companies such as Apple and Qualcomm mobile phone chips, and "smart" devices like household appliances. However, this has all begun changing very fast in recent years (Woo, 2020).

For example, Apple recently switched over from Intel x86 chips to power their Macbooks and are now exclusively using ARM chips rebranded as M1, M2, and M3 cores (Shilov, 2023). Amazon has begun designing in-house chips called Graviton (Morgan, 2022) to power their datacenter and compute services. Furthermore, Microsoft has also been adopting their operating system design to support ARM, both for in-house hardware (such as Surface tablets) as well as general purpose compute (for Windows OS). While many business decisions go into such a drastic change in infrastructure, the common thread is that ARM-based devices provide excellent energy-efficiency compared to their x86 counterparts, and have dramatically bridged the gap in performance as well (CloudPanel, 2023).

Despite lots of potential advantages of low-powered ARM processors, adopting or moving to a new ISA in the hardware space renders prior code in the software space incompatible, as the machine code must be re-targeted for the new ISA. The ISA defines the hardware-software "contract",

* These authors contributed equally.

which has enabled substantial improvements over the years independently in the software space and hardware space. Nevertheless, an alternative "machine language" presents a huge technological barrier to overcome, as code portability now becomes a problem. Additionally, for competitive reasons, many companies do not ship their source code around, and instead provide executable binaries which are challenging to de-compile and break intellectual property (IP).

Alternatively, to avoid having to recompile a huge code base and to address code portability challenges, hardware virtualization has been used to dynamically "translate" between ISAs. Several tools currently facilitate x86 to ARM virtualization. QEMU (Bellard, 2005b), an open-source, general-purpose emulator supporting multiple architectures and, while versatile, introduces significant performance overhead compared to native execution (Wei et al., 2019). Rosetta 2 (Apple Inc., 2020), Apple's proprietary translation layer, specifically designed for x86 to ARM (Apple Silicon) translation, is more efficient than QEMU, but its closed-source nature and platform-specificity limit its broader application. Given these existing solutions' limitations, a crucial question emerges: Can we develop a direct translation medium between x86 and ARM architectures that ensures correctness without the performance hit of a virtualization layer?

Addressing the *correctness* of assembly to assembly translations is arguably the more challenging research aspect.

In addition to legacy software compatibility reasons, CISC instructions have fundamental architectural differences to RISC instructions, such as how an instruction handles registers versus memory for different opcodes, the number of available general purpose registers between the ISAs, and the access of sub-register bits (e.g., modifying only the lower 8-bits of a register value). Similarly, compilers need to work harder to generate ARM-based code, while that complexity is lowered onto the micro-architecture in an x86-based system. Binary sizes are also different, where programs assembled into x86 are typically much shorter (due to the complex nature of instructions which are later converted to micro-code inside the processor), compared to the longer binaries in ARM (composed of many, simpler instructions). Furthermore, operating systems need to be aware of the underlying "language" of the hardware in order to properly manage it; we observe this phenomenon when seeing that Windows today has two separate OS binaries for x86 versus ARM (Anderson & Smith, 2023; Microsoft, 2024).

Existing approaches demonstrate an apparent dichotomy: open-source emulators prioritize flexibility over performance, while solutions like Rosetta achieve efficiency through hardware-specific optimizations. This creates a gap in the ecosystem for a solution that combines both attributes.

This paper proposes an approach based on language models (LMs) as machine translation engines. Building upon LMs' demonstrated success in various translation tasks, we apply them to learn mappings between x86 and ARM assembly code through paired examples. Intuitively, we use the LM to "translate" between CISC and RISC machine code, just as it has performed admirably in recent years in enabling translation between human languages. A key difference, however, is that the translation must be *precise* in machine languages, as any incorrect syntax, mis-used registers, bad jumps, or other architecturally important constructs can render the program incorrect semantically and/or functionally. Our methodology presents an opportunity to merge open development practices with high-performance translation capabilities, as the models can identify and optimize instruction patterns without depending on proprietary optimizations or predetermined translation rules while avoiding complete system emulation overhead or code overhauls.

This work presents the following key contributions:

1. The first CISC to RISC transpiler, coined CRT, built via a custom-trained LLM achieving a test accuracy of 79.25% on ARM and 88.69% on RISC-V64.
2. An in-depth analysis into the inner workings of our transpiler, including hardware-informed design decisions to best train an accurate LLM model for assembly transpilation (§3, §6).
3. We perform a case-study using our transpiler in a real-world setting, and compare it to Apple Rosetta's x86 to ARM virtualization engine. Results show that CRT's generated assembly achieves 1.73x speedup compared to Rosetta while delivering 1.47x better energy efficiency and 2.41x memory efficiency (§7).

In the remainder of this paper, we provide additional background in the space of assembly languages and LLM-for-hardware (§2), describe our approach in designing the transpiler (§3), evaluation of its efficacy (§4), and discuss the challenges and benefits of our approach (§5, §6). Finally, we conclude with a case study of transpiling for an Apple M2 processor, and the advantages over prior approaches for this task (§7).

2 Background & Related Work

Virtualization and Emulation: Emulation and assembly-level virtualization enable the execution of one ISA's binary on a host machine for which it was not compiled for originally. QEMU (Bellard, 2005a), an open-source emulator, uses dynamic binary translation (Sites et al., 1993), offering flexibility but with performance overhead, enabling x86 to ARM emulation, amongst other ISAs. Rosetta 2, Apple's virtualization layer for macOS, combines

ahead-of-time (AOT) and just-in-time (JIT) translation, providing better performance within the Apple ecosystem.

These approaches face challenges in achieving native-level performance and ensuring broad compatibility, due to the dynamic nature of execution. A transpiler approach, directly converting x86 to ARM assembly, could supplant these solutions by eliminating runtime translation overhead with a one-time translation into the host ISA. This method could address the limitations of current emulation and virtualization techniques, particularly in performance-critical scenarios, or where pre-processing is feasible, or when source code is not available (due to proprietary IP).

Neural Code Translation: Machine learning approaches for code translation have primarily focused on high-level programming. Initial research explored neural machine translation architectures, with TransCoder (Lachaux et al., 2020) demonstrating unsupervised translation between C++, Java, and Python. Pre-trained transformer models such as CodeBERT (Feng et al., 2020) and CodeT5 (Wang et al., 2021) have advanced code understanding and generation capabilities across multiple programming languages. The emergence of LLMs specialized for code, including Code Llama (Roziere et al., 2023) and DeepSeek Coder (Liu et al., 2024), has demonstrated increasingly sophisticated code manipulation capabilities through self-supervised learning on vast code repositories. These methods often rely on structural representations like abstract syntax trees for high-level language translation (Chen et al., 2018). However, these methods face distinct challenges when applied to assembly-level translation, where the structural representations and semantic preservation requirements differ significantly from high-level language translation.

Language Models for Low-Level Programming: Recent research has increasingly demonstrated the potential of language models in various tasks related to low-level code analysis and transformation. A language model is a statistical model that learns to predict the probability distribution of tokens within a language, enabling it to generate coherent and contextually relevant text. These models have been successfully applied in areas such as decompilation, binary similarity analysis, and compiler optimization, demonstrating their ability to tackle intricate, instruction-level challenges.

In decompilation, LLM4Decompile (Tan et al., 2024) introduced specialized language models for direct binary-to-source translation and decompiler output refinement. DeGPT (Hu et al., 2024) further explored decompiler enhancement through semantic-preserving transformations. SLaDe (Armengol-Estapé et al., 2024) combines a 200M-parameter sequence-to-sequence Transformer with type inference techniques to create a hybrid decompiler capable of translating both x86 and ARM assembly code into readable and accurate C code, effectively handling various optimization levels (`-O0` and `-O3`).

Language models have also been adapted to optimization tasks, with LLM Compiler (Cummins et al., 2024) introducing a foundation model that supports zero-shot optimization flag prediction, bidirectional assembly-IR translation, and compiler behavior emulation. This approach demonstrates the potential for language models to enhance compiler optimization workflows by automating complex tasks.

Binary similarity analysis has similarly benefited from language model adaptations. DiEmph (Xu et al., 2023) addressed compiler-induced biases in transformer models, while jTrans (Wang et al., 2022) incorporated control flow information into the transformer architecture. Yu et al. (Yu et al., 2020) combined BERT-based semantic analysis with graph neural networks to capture both semantic and structural properties of binary code.

While these applications have shown promising results, the use of LLMs for assembly code transpilation remains relatively underexplored. Assembly languages present unique challenges due to the fundamental differences in instruction sets and execution models across architectures. GUESS & SKETCH (Lee et al., 2024) introduced a neurosymbolic approach combining language models with symbolic reasoning for translating assembly code between ARMv8 and RISC-V architectures. Our work, CRT, extends this direction by addressing the more challenging task of transpiling between CISC (x86) and RISC (ARM, RISC-V) architectures, bridging fundamental architectural differences in ISA complexity and execution models.

3 Approach

This sections describe our approach for CISC to RISC assembly transpilation (CRT), covering the problem setup, model choices, training, and tokenizer adaptations.

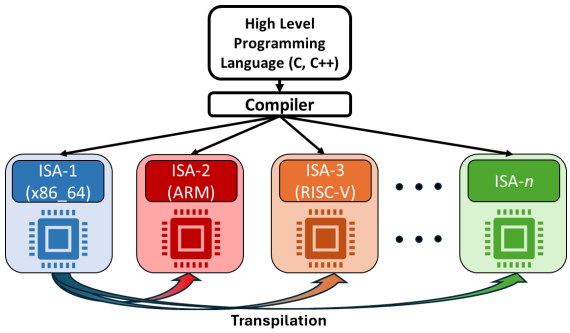


Figure 1. Conceptual representation of an asm-to-asm transpiler, which would enable direct "translation" from one machine language to another without needing the source code and by-passing the software stack.

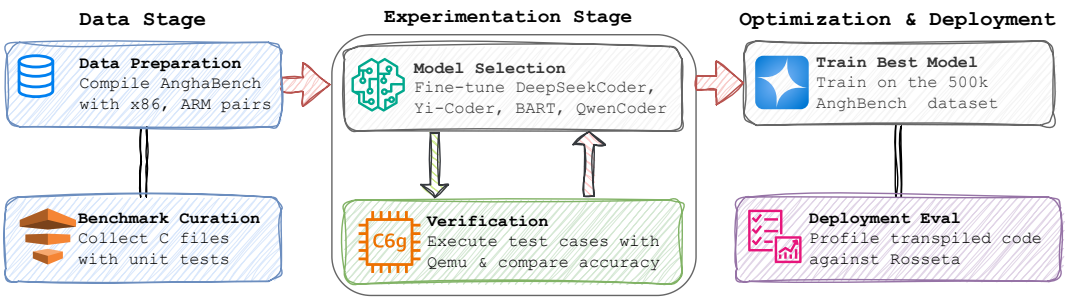


Figure 2. CRT pipeline stages: **Data** (AnghaBench data curation), **Experimentation** (model tuning and accuracy), and **Optimization & Deployment** (final training and Rosetta evaluation).

3.1 Problem Definition

We aim to translate x86 assembly code to ARM assembly code by leveraging Language Models (LMs) to automatically handle the fundamental differences between these ISAs. Let $X = \{x_1, x_2, \ldots, x_n\}$ denote the set of x86 vocab, and $Y = \{y_1, y_2, \ldots, y_n\}$ denote the set of corresponding ARM vocab. Our goal is to learn a mapping function $f : X \rightarrow Y$ that translates any x86 code x_i into its ARM equivalent y_i.

Our approach is to utilize LLMs and have the model learn the conditional distribution $P(Y \mid X; \theta)$, where θ represents the model parameters. The model generates ARM code in an autoregressive manner, producing each token y_t based on the input x86 code X and previously generated tokens $y_{<t}$: $P(Y \mid X; \theta) = \prod_{t=1}^{T} P(y_t \mid y_{<t}, X; \theta)$. This approach aims to provide contextually accurate translations, avoiding prior rule-based methods of the past (Ebcioğlu & Altman, 1997).

3.2 Training Stages and Model Selection

Our framework consists of three main stages: Data collection, Experimentation (of hyperparameters), and Optimization & Deployment, as illustrated in Figure 2.

Data Stage: We construct our dataset by collecting a diverse corpus of C source files and generating corresponding x86 (CISC) and ARM (RISC) assembly pairs. These paired compilations constitute our training data, from which the language model learns the cross-architectural mapping between CISC and RISC instruction sets. For evaluation, we employ two metrics: functional correctness assessed through unit tests accompanying the C files, and assembly-level similarity measured via the more strict metric of *edit distance* (§4.4) between the generated outputs and ground truth assembly pairs.

Experimentation Stage: For model selection, we fine-tune various small-scale, established open-source language models, leveraging their efficiency in processing high-level code. During this phase, we experiment with hyperparameters including batch size, gradient accumulation (Lamy-Poirier, 2021), warmup steps (Goyal, 2017), optimizers (e.g., AdamW (Loshchilov, 2017) and Adafactor (Shazeer & Stern, 2018)), learning rates, and epochs to find the best settings for our application of assembly transpilation. Each model undergoes a thorough evaluation on our curated benchmark, and accuracy is verified by executing test cases using Qemu (Bellard, 2005b) to ensure reliability. We also explored the alignment of code pairs in this phase. While this showed initial promise, it did not help significantly and so we leave our experimentation details for the interested reader in appendix §A.3.

Optimization & Deployment: In the stage, the best-performing model is selected and trained on the entire dataset. We also explore post-training quantization, to study the efficacy of compressing our asm-to-asm transpiler for a low-resource deployment setting. We run a case study to evaluate and compare the transpiled code against Rosetta (Apple Inc., 2020), to assess performance in a production-like environment (§7).

3.3 Tokenizer Extension

Input	`ldr r1, r2`
Tokenizer	**Tokens**
DeepSeek/Yi-Coder	`ld` `r` `␣` `r` `1` `,` `␣` `r` `2`
Our Extended Tokenizer	`ldr` `␣` `r1` `,` `␣` `r2`

Table 1. Comparison of tokenization approaches between DeepSeek/Yi-Coder and our extended tokenizer. Spaces are represented as ␣ and shown with colored backgrounds to highlight token boundaries. Note how our tokenizer groups related tokens (e.g., `ldr` and `r1`) as singular units.

To enhance our LLMs' understanding and generation of assembly code, we extended the tokenizer to include the most frequently used opcodes and register names from both x86 and ARM architectures (see Table 1). Tokenization segments raw text into tokens for the model to process (Vaswani, 2017), allowing accurate recognition

of assembly language components. This customization efficiently represents the distinct semantics of each instruction set, improving the model's ability to parse and generate correct translations by aligning with the low-level details of the input assembly code.

4 Experiments and Evaluation

We present a comprehensive evaluation of CRT's effectiveness in x86-to-ARM binary transpilation across three dimensions: training data preparation (§4.1), hyperparameter tuning (§4.2), and model architecture selection. Using standard benchmarks (§4.3) and metrics (§4.4), we assess the generated assembly code's semantic preservation and functional correctness.

4.1 Training Data

The training dataset was derived from AnghaBench (Da Silva et al., 2021), a comprehensive benchmark suite containing 1 million compilable C programs mined from major public C repositories on GitHub. From this benchmark, we randomly sampled 500k programs to form our training set, equivalent to 8 billion tokens. These programs were then compiled to x86 using `gcc` (Compiler, 2009) and cross-compiled to ARMv5 using `ARM-gnueabi-gcc` (Radcolor, n.d.); both sets were generated on an AMD Ryzen 7 processor.

4.2 Experimental Setup

All of our hyperparameter optimization experiments were conducted on a small 100k portion of AnghaBench. We tested various hyperparameter settings on this subset of our benchmark. After identifying the optimal configuration, we scaled up the training data to 500k samples. We trained three models: DeepSeek-Coder1.3B (Guo et al., 2024), Yi-Coder2B (01.AI, 2024), and BART-Large (300M) (Lewis, 2019) on the AnghaBench dataset. Given the dataset size of 1 million samples, with an average of 13k tokens per sample, we opted for smaller models and worked with reduced dataset size of 500k samples. All models were trained using four A100 GPUs (40 GB each). Training with 500k samples, a batch size of 4, and 2 epochs required three days. To conserve memory, mixed precision training with `bfloat16` was employed. Given limited capacity for large batch sizes, we applied gradient accumulation (Lamy-Poirier, 2021) with an effective batch size of 4. Additionally, all models were trained with optimization level `-O0`, and we used paged AdamW (Loshchilov, 2017) to avoid memory spikes, with a weight decay of 0.001. We chose a small learning rate of 1×10^{-4} with a linear schedule, as experiments indicated this schedule performed best. All models were trained with a context window of 16k. For inference, we used caching to enhance inference speed and disabled sampling to ensure deterministic outputs. Our evaluation set was sourced from the LLM4Decompile (Tan et al., 2024) set, compiled to x86. We employed QEMU (Bellard, 2005b) to simulate the evaluation environment. Following training, we apply quantization techniques (e.g., `bfloat16`, `int8`, and `int4`) using `llama.cpp` (Ggerganov) to optimize for efficient inference on CPU-based devices. This step is crucial to maintain high performance while reducing the computational load, making the solution feasible for local deployment.

4.3 Evaluation Benchmark

CRT's performance accuracy is evaluated using the HumanEval benchmark, originally introduced by (Chen et al., 2021) for Python code generation. The benchmark consists of 164 programming problems that assess language comprehension, reasoning, and algorithmic thinking. For our evaluation, we utilize the C-translated version from LLM4Decompile (Tan et al., 2024), which maintains the same problems while converting both function implementations and test cases to C code.

To ensure thorough testing, we measure code line coverage using `gcov`, GNU's source code coverage analysis tool. As emphasized by (Myers et al., 2011), line coverage is a fundamental metric in software testing that indicates which lines of code were executed at least once during testing, helping identify untested code paths and potential blind spots in test suites. The higher the line coverage percentage, the more comprehensive the testing. HumanEval resulted in an average line coverage of 98.81%, indicating that nearly all lines of code were executed during testing.

For the evaluation process, we generate the corresponding assembly code pairs following the training data preparation process detailed in Section §4.1.

4.4 Evaluation Metrics

We evaluate our approach using two primary metrics:

Edit Distance: Following prior work (Lee et al., 2024), we employ the Levenshtein edit distance (Lcvenshtcin, 1966) between the ground truth and transpiled ARM assembly as an initial measure of syntactic similarity. However, this metric is limited because semantically equivalent assembly sequences can differ syntactically due to variations like register allocation or instruction ordering (§6).

Functional Correctness: To overcome the limitations of edit distance, we assess the functional equivalence by running the ground truth and transpiled code against comprehensive test suites, employing software testing principles via unit test coverage. A transpilation is considered correct if it passes all test cases of the corresponding program. The

Model	Average Edit Distance (↓)	Exact Match (↑)	Test Accuracy (↑)
GPT4o (OpenAI, 2024)	1296	0%	8.18%
DeepSeekCoder2-16B (Zhu et al., 2024)	1633	0%	7.36%
Yi-Coder-9B (01.AI, 2024)	1653	0%	6.33%
Yi-coder-1.5B	275	16.98%	49.69%
DeepSeekCoder-1.3B	107	45.91%	77.23%
DeepSeekCoder-1.3B-xTokenizer-int4	119	46.54%	72.96%
DeepSeekCoder-1.3B-xTokenizer-int8	**96**	49.69%	75.47%
DeepSeekCoder-1.3B-xTokenizer	165	**50.32%**	**79.25%**

Table 2. Comparison of models' performance on the *x86 to ARM* transpilation task, measured by Edit Distance, Exact Match, and Test Accuracy. The top portion lists pre-existing models, while the bottom portion lists models trained by us. Arrows (↑, ↓) indicate whether higher or lower values are better for each metric. The best results are highlighted in **bold**.

experimental results presented in §5 show that a significant portion of the transpilation results differ syntactically (having non-zero edit distance) from the ground truth yet preserve program semantics and execute correctly.

5 Results

We evaluate the efficacy of our transpiler for CISC-to-RISC assembly translation, focusing on the correctness of the output ARM assembly. Utilizing the metrics defined above (§4), we compare our approach with state-of-the-art coding LLMs and evaluate our approach for x86 to ARM transpilation (Table2).

5.1 Transpiler Validation

Of the evaluated LLMs, DeepSeekCoder-1.3B-xTokenizer (our model utilizing the tokenizer and trained as described in §4.2) achieves a test accuracy of **79.25%** on x86 to ARM transpilation, substantially outperforming larger models such as GPT4o (8.18%), DeepSeekCoder2-16B (7.36%), and Yi-Coder-9B (6.33%) (Table 2). Despite being 9× to 20× smaller in size, our model exhibits at up to 9.8× the accuracy, highlighting the effectiveness of our approach.

Compared to prior work on ARM to RISC-V translation (Lee et al., 2024) (which achieves 68% accuracy - 80% when using symbolic analysis - with a similarly sized model), our model attains higher accuracy on the more complex task of x86 to ARM translation without the use of external, symbolic testing or tools. We attribute this approximate 7.8% difference to our better tokenization scheme (which is needed for the CISC/RISC disambiguation) as well as the 8x larger context window.

An interesting comparison arises between DeepSeekCoder-1.3B and Yi-Coder-1.5B; DeepSeekCoder outperforms Yi-Coder by 28%. We attribute this to DeepSeekCoder being trained from scratch on 2 trillion code tokens, whereas Yi-Coder is based on the Yi-chat model, potentially limiting its efficacy for this specific task.

Additionally, the results from our aligned BART shows a 12% jump in accuracy from baseline BART inspired by (Lee et al., 2024). This shows that our alignment technique is effective even with a very low context window of 1024 but yet still lacks behind large context window of DeepSeekCoder by 62.8%.

Our model exhibits strong syntactic robustness, as evidenced by the high number of transpiled programs with an edit distance of zero (Table 2, column 3). This indicates that the generated assembly code is often syntactically identical to the reference code, demonstrating proficiency in producing correct assembly language syntax for both RISC architectures (ARM & RISC-V). Moreover, the absence of syntax errors allows us to focus our evaluation on functional correctness, measured by test accuracy metrics. This aligns with our observation that LMs rarely produce typos or grammatical errors, even in assembly instructions, based on our tests.

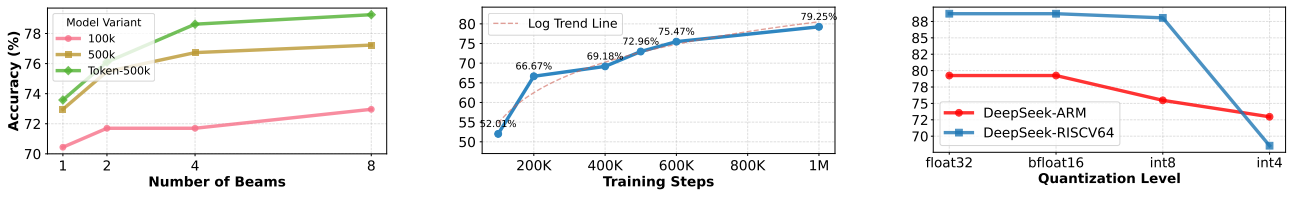
5.2 Impact of Tokenizer and Quantization

Enhancements in our tokenizer contribute to the model's accuracy. DeepSeekCoder-1.3B-xTokenizer increases accuracy by 2% compared to DeepSeekCoder-1.3B, indicating that our extended tokenizer enables more efficient learning. Additionally, the optimized tokenizer reduces the average number of tokens by 7.27%, improving inference speed.

Our models quantized to `int8` and `int4` precision achieve comparable results to the `float32` models. For ARM transpilation, accuracy decreases by only 3.8% when moving from `float32` to `int8`, and by nearly 2.5% when moving to `int4` (Table 2 column 2).

5.3 Training Performance

Selecting appropriate training hyperparameters has a significant impact on model accuracy and edit distance. In our experiments, implementing warmup steps (Goyal, 2017)



(a) (b) (c)

Figure 3. DeepSeek-1.3B performance: (a) Accuracy across beam sizes (1, 2, 4, 8) for different training data sizes. (b) Accuracy progression over training steps with a logarithmic trend. (c) Quantization impact (float32, bfloat16, int8, int4) on ARM and RISC-V64.

notably improved the model's edit distance, decreasing it by 8%. This improvement is attributed to enhanced convergence and training stability, which are crucial for complex tasks like x86 to ARM transpilation. Since we are fine-tuning models that were initially trained to follow specific instructions to produce code, the use of warmup steps with a higher initial learning rate helps our model transition from this instruction-based generation setting to our transpilation setting, thereby avoiding early overfitting and increasing generalization (Kalra & Barkeshli, 2024).

5.4 Inference Performance

As shown in Figure 3, we analyze the model's accuracy across different inference configurations. Our beam search experiments (Figure 3a) demonstrate that increasing the number of beams improves accuracy. Specifically, as the beam size increases from 1 to 8, the model explores multiple decoding paths, akin to human-like consideration of different options. This approach is similar to the symbolic solving post-processing technique used in (Lee et al., 2024), but with a computational complexity of $O(bN^2)$ compared to their $O(2^N)$ complexity. This increased flexibility allows the model to produce more robust solutions. However, larger beam sizes increase computational overhead, presenting a trade-off between performance and inference speed.

From a runtime performance, our models exhibit real-world inference efficiency. On an NVIDIA A100 GPU with 15.6 TFLOPS, our model generates a sample with a 16k context length in 18.3 seconds on average, achieving a rate of 437.2 tokens per second. On a Ryzen 7 CPU, the model achieves 18.51 tokens per second with `int8` quantization and 87.23 tokens per second with `int4` quantization using Ollama, demonstrating applicability for real-world use cases requiring efficient inference.

Figure 3b shows that our model's accuracy increases logarithmically with the size of the training dataset. As more data is provided, the model better learns to reason about transpilation with diverse training examples following LLMs scaling laws (Kaplan et al., 2020).

We also experimented with different compilers (GCC and Clang) during training and found that both produced similar results, with an edit distance of 133 and exact match accuracy of 50.31%. All results are thus reported with the GCC compiler, due to its compatibility with QEMU for running tests as mentioned in section 4.2.

5.5 Analysis of Transpiled Assembly Code

To better understand the transpilations generated by CRT, we examined the evaluation benchmark's ground truth and compared it with the transpiled code. An interesting case arises when the transpiled code is correct, but the edit distance from the ground truth deviates from zero.

Our investigation revealed various patterns of implementation that maintain functional correctness despite differing from the ground truth. Commutative operations, like the `add` opcode, sometimes show order differences between the ground truth and predicted code. Register allocation variations occur when different registers are chosen for the same operations, as long as data flow and register dependencies are preserved. Memory location swapping is another pattern where variables are stored in different memory locations, yet their relationships remain consistent. Additionally, stack frame sizes sometimes differ between the ground truth and transpiled code.

We also identified cases of instruction-level semantic equivalence, where different instruction sequences achieve the same logical result. For example, multiplication in the ground truth might use a direct `mul` opcode, while the transpiled code achieves the same computation using shifts and additions. Variations in constant handling were also noted, with the transpiled code sometimes using immediate values directly in instructions while the ground truth loads constants from memory. Instruction consolidation is common, with the transpiled code combining multiple instructions (`mov r1, r2`; `add r1, r1, #1`) into single, streamlined versions (`add r2, r2, #1`) that maintains functionality. In many instances, these patterns appear together in what we refer to as "composite variations".

The error patterns become particularly evident in assembly code, where even slight variations can lead to significant functional issues. For instance, non-commutative operations are prone to critical errors when operand order is reversed,

Model	Average Edit Distance (↓)	Exact Match (↑)	Test Accuracy (↑)
GPT4o (OpenAI, 2024)	1293	0%	7.55%
DeepSeekCoder2-16B (Zhu et al., 2024)	1483	0%	6.29%
DeepSeekCoder-1.3B-xTokenizer-int4	112	14.47%	68.55%
DeepSeekCoder-1.3B-xTokenizer-int8	31	69.81%	88.05%
DeepSeekCoder-1.3B-xTokenizer	**27**	**69.81%**	**88.68%**

Table 3. Comparison of models' performance on the *x86 to RISCv64* transpilation task.

as seen in subtraction, where using `sub r1, r3, r2` instead of `sub r1, r2, r3` completely changes the computation. Incorrect register management, where registers are overwritten prematurely, can lead to data loss. Immediate value errors also appear, particularly in shift operations; for instance, `asr r2, r2, #1` is mistakenly used for division by 2 when division by 4 (`asr r2, r2, #2`) is needed, causing computational discrepancies. Memory addressing errors, such as misaligned access, occur when values are stored and retrieved from incorrect offsets, resulting in data corruption. Additional examples are provided in the Appendix§A.1.

We also observe for incorrect cases with a high edit distance, the transpiled code diverges significantly from the ground truth, often reflecting substantial deviations from the intended logic—sometimes even resulting in unintended behaviors like infinite loops. Such drastic variations make it difficult to interpret or troubleshoot the transpiled output by simply comparing it to the ground truth, as the underlying logic no longer aligns.

5.6 Extension to RISC-V ISA

To demonstrate the generality of our method, we also trained our model on the task of transpiling from x86 to RISC-V64, achieving a test accuracy of **88.68%** (Table 3). Notably, our model significantly outperforms existing models like GPT4o and DeepSeekCoder2-16B, which achieved much lower test accuracies of 7.55% and 6.29%, respectively. However, this result indicates that transpiling to RISC-V64 is easier than to ARM, possibly due to RISC-V's simpler and more consistent instruction set compared to the more complex instructions in recent ARM architectures. For RISC-V efficient transpilation (Table 3 column 2), the `int8` quantized model's accuracy is nearly identical to the `float32` model, with a minor drop of 0.63% and the same exact match accuracy. However, the `int4` quantized model experiences a significant accuracy drop of 19.5%. The results not only reinforce that we can deploy our model on consumer-grade machines with high-efficiency, but also from an information theory perspective that the number bits necessary for CISC-to-RISC transpilation can still accurately encode functional semantics for machine code.

6 Discussion

Our proposed model effectively tackles the challenges of CISC-to-RISC assembly code transpilation, achieving high performance without extensive model scaling. In this section, we provide insights into our model's behavior and the factors influencing its performance.

One key observation is the performance difference between x86-to-ARM and x86-to-RISC-V64 transpilation. Our model performs better when translating to RISC-V, likely due to its simpler instruction set (47 instructions) compared to ARM's more complex set (approximately 100 instructions), as shown in Figure 6. This suggests that the complexity of the target ISA significantly affects the difficulty of the transpilation task. Consequently, we focused most of our experiments on x86-to-ARM transpilation to address challenges associated with more complex RISC ISAs.

Another important aspect is the model's syntactic flexibility. While CRT achieves high functional correctness, it often produces syntactically diverse outputs, as indicated by the high edit distance metric. Instead of memorizing patterns, the model reasons through register mappings and instruction sequences, generating functionally correct but syntactically varied code. This flexibility is beneficial in real-world code generation, where functional equivalence is more valuable than syntactic similarity. For example, the model may choose different sets of registers for each instance, as long as it maintains consistency in their usage. Acceptable variations include using different operand orders in commutative instructions (e.g., `add r0, r1, r2` vs. `add r0, r2, r1`, assigning temporary variables to different registers, combining multiple instructions into a more efficient single instruction, varying allocated memory size or resource usage without affecting functionality, employing different methods for loading constants, and consistently swapping memory locations for variables (e.g., using `[fp, #-8]` instead of `[fp, #-12]` for a loop counter). These variations showcase the model's ability to generate functionally equivalent but syntactically diverse code, which is often desirable in assembly code generation.

Despite these successes, CRT is not without limitations. We observed that certain errors persist, which can be

categorized into three main types (1) **Register Allocation Errors (14.11%):** Allocating registers that have already been allocated, leading to memory issues (2) **Addressing Errors (62.03%):** Jumping to prohibited memory addresses or copying incorrect addresses from x86 code (3) **Other Errors (23.86%):** Including invalid constants and floating-point exceptions. We noticed that additional training reduced the number of Type 1 errors by 21.45%, Type 2 errors by 7.67%, and Type 3 errors by 1.39%. This suggests that while increased training data can mitigate some errors, particularly those related to register allocation, other errors may require different strategies.

One of the challenges we identified is related to the model's handling of numerical tokens. Current tokenizers often treat each digit of a number as a separate token, which can hinder the model's ability to correctly process long numerical values, such as memory addresses or constants that need to be accurately copied or slightly modified from the x86 code. We believe that developing a better tokenizer that handles numbers as single tokens could significantly improve the model's performance, particularly in reducing addressing and constant-related errors.

Notably, our model, despite being at least seven times smaller than those used in similar studies (Lee et al., 2024; Tan et al., 2024), was effective in this task. This suggests that for machine language translation, model quality and data curation are more critical than sheer model size. As shown in Figure 3b, the quality and size of the training data significantly impact performance. Additionally, using an extended tokenizer improved the understanding of instructions and registers, and a longer context window enabled the model to track register usage effectively. Achieving these results with a small, quantized model indicates that efficiency and practicality need not be sacrificed for performance. This has significant implications for deploying such models on resource-constrained hardware common in embedded systems and other RISC applications.

Looking ahead, there are several avenues for improving our model. In addition to developing a better tokenizer, as previously mentioned, incorporating more diverse and comprehensive training data could further reduce errors, particularly those related to addressing and constants. Exploring techniques such as incorporating domain-specific knowledge or constraints into the model training process might also enhance performance.

Overall, our findings highlight that high-quality assembly code transpilation across diverse ISAs requires a holistic approach that goes beyond merely increasing model size. Thoughtful design of tokenization and training processes, attention to ISA-specific challenges, and efficient quantization collectively enable high-performing, deployment-ready models. These insights contribute to a deeper understanding of how models can be designed to facilitate the industry's ongoing transition towards scalable, energy-efficient processor architectures, supporting software compatibility and performance across platforms.

7 Case Study

To evaluate the efficiency of our transpiler, we performed a real world study on an Apple M2 Pro machine, which features a more recent version of ARM, the ARM64v8-A architecture. This transition offered two key advantages: first, it enabled us to use the native ARM compiler toolchain to generate the target instruction sequences, rather than relying on cross-compilation; second, through Apple's Rosetta 2 translation layer, we established a unified testing environment enabling comparative analysis across different execution modes within the same hardware platform.

7.1 Experimental Setup

The evaluations were conducted using Apple Clang 14.0.3 (clang-1403.0.22.14.1) on an Apple M2 Pro processor with 16GB of RAM running macOS 13.7. All programs were compiled targeting the arm64-apple-darwin22.6.0 architecture with `-O0` optimization level. Additional details are the same as §4.2 for our transpilation correction evaluation.

We examine the execution characteristics across three distinct execution environments on the M2 Pro machine. First, we establish the baseline performance by executing natively compiled ARM64 binaries. Second, we measure the performance of x86 binaries executed through Apple's Rosetta 2 dynamic binary translation layer. Third, we evaluate our CRT transpiled code that directly transforms x86 assembly to ARM64. For each environment, we analyze three performance dimensions: execution time, CPU energy consumption measured using powermetrics instrumentation, and memory utilization patterns. To ensure statistical validity, we execute each program 100 times and compute its geometric mean for reporting (Fleming & Wallace, 1986). All performance evaluations are conducted under controlled conditions.

The functional correctness of the proposed approach, when trained on ARMv8, yielded the results shown in Table 4.

Model	**AED (↓)**	**EM (↑)**	**Acc. (↑)**
CRT ARMv5	165	50.32%	79.25%
CRT ARMv8	105	50.61%	75.0%

Table 4. Correctness comparison between CRT implementations on ARMv5 and ARMv8 architectures. Metrics include Average Edit Distance (AED), Exact Match (EM), and Test Accuracy (Acc.).

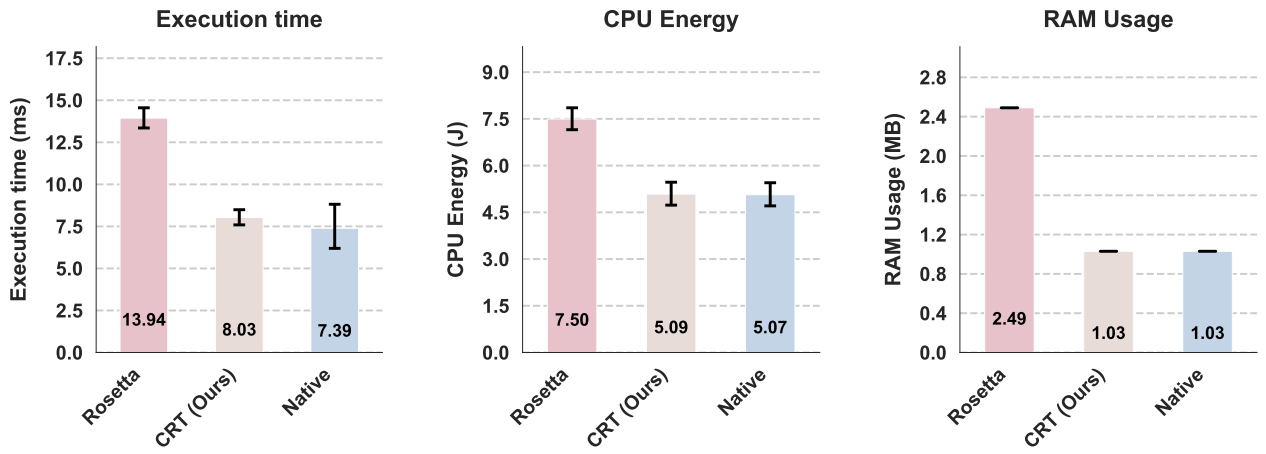


Figure 4. Measured execution time, energy utilization, and RAM usage across different settings on Apple M2 Macbook.

A key finding is that compared to ARMv5, we observed a performance drop of 4.25% with ARMv8 achieving 75.0% on the evaluation-set. This decline can be attributed to ARMv8's increased architectural complexity (refer to Appendix A.2). The increased complexity is evident in several areas, including register usage patterns, addressing modes, comparison operations, and floating-point handling. In each of these areas, ARMv8 adopts more sophisticated approaches compared to ARMv5's straightforward implementations. These architectural features make instruction patterns more challenging for the LLM to learn effectively.

M2 (ARMv8) \ QEMU (ARMv5)	Correct	Incorrect
Correct	**108** *(65.9%)*	**15** *(9.1%)*
Incorrect	**23** *(14.0%)*	**18** *(11.0%)*

Figure 5. Confusion matrix of the proposed approach executed on QEMU (ARMv5) and M2 (ARMv8) for HumanEval programs.

Examining the confusion matrix (Figure 5) for the proposed approach's performance across ARMv5 and ARMv8 architectures reveals an agreement of 76.8%. However, we observe distinct failure patterns (15 versus 23 cases failing uniquely on ARMv5 and ARMv8 respectively), suggesting architecture-specific performance variations that induce divergent error modes in the LLM's execution.

7.2 Performance Analysis

The performance evaluation shows that CRT achieves near-native efficiency across multiple metrics. With execution times nearly matching native code, CRT delivers a 1.73x speedup over Rosetta, along with 1.47x better energy efficiency and 2.41x better memory efficiency. Notably, CRT's memory footprint remains close to native execution (1.034 MB vs. 1.03 MB), whereas Rosetta requires 2.49 MB.

7.3 ARMv5 versus ARMv8 Analysis

The proposed transpiler enables efficient cross-architecture code execution with performance nearing native compilation, proving the feasibility of LLM-based binary translation. These results indicate that machine learning approaches can effectively learn complex mappings between instruction set architectures, maintaining high performance with significant improvements over Rosetta.

Looking forward, this opens the door for efficient CISC to RISC transitions, particularly to ARM architectures, while maintaining seamless control over legacy x86 software with reduced overhead and improved performance. This could potentially accelerate the adoption of ARM architectures across enterprise environments.

8 Conclusion

This paper presents CRT, a language model-guided transpiler for converting CISC (x86) assembly code to RISC (ARM and RISC-V) architectures. Our work explores a shift in systems software design - moving away from hand-crafted rules and predetermined translation patterns toward more intelligent and adaptable solutions. As hardware architectures continue to diversify and evolve, particularly with the rise of domain-specific accelerators, learning-based approaches may become increasingly relevant for maintaining software portability while maximizing hardware performance. The future of systems software lies in leveraging machine learning to create solutions that can adapt to architectural changes without requiring separate engineering efforts for each new target platform.

REFERENCES

01.AI. Meet yi-coder: A small but mighty llm for code. https://huggingface.co/blog/lorinma/yi-coder, 2024. Published September 4, 2024.

Anderson, A. and Smith, K. Windows on ARM64: Architecture, implementation, and performance analysis. In *USENIX Annual Technical Conference*, pp. 223–238. USENIX Association, 2023.

Apple Inc. Apple's rosetta 2 overview, 2020. URL https://support.apple.com/en-us/HT211861. Accessed: 2024-10-31.

Armengol-Estapé, J., Woodruff, J., Cummins, C., and O'Boyle, M. F. SLaDe: A Portable Small Language Model Decompiler for Optimized Assembly. In *2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2024.

Bellard, F. QEMU, a fast and portable dynamic translator. In *USENIX Annual Technical Conference, FREENIX Track*, 2005a.

Bellard, F. Qemu, a fast and portable dynamic translator. In *USENIX Annual Technical Conference, FREENIX Track*, 2005b. URL https://www.qemu.org/.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Chen, X., Liu, C., and Song, D. Tree-to-tree neural networks for program translation. *Advances in neural information processing systems*, 2018.

CloudPanel. What are arm-based servers? comparison with x86, benefits and drawbacks, 2023. URL https://www.cloudpanel.io/blog/arm-based-servers/. Accessed: 2024-10-31.

Compiler, G. The gnu compiler collection. *http://gcc. gnu. org/-Acesso em*, 10(11):2009, 2009.

Cummins, C., Seeker, V., Grubisic, D., Roziere, B., Gehring, J., Synnaeve, G., and Leather, H. Meta large language model compiler: Foundation models of compiler optimization. *arXiv preprint arXiv:2407.02524*, 2024.

Da Silva, A. F., Kind, B. C., de Souza Magalhães, J. W., Rocha, J. N., Guimaraes, B. C. F., and Pereira, F. M. Q. Anghabench: A suite with one million compilable c benchmarks for code-size reduction. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pp. 378–390. IEEE, 2021.

Ebcioğlu, K. and Altman, E. R. Daisy: dynamic compilation for 100% architectural compatibility. ISCA '97. Association for Computing Machinery, 1997. URL https://doi.org/10.1145/264107.264126.

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*, 2020.

Fleming, P. J. and Wallace, J. J. How not to lie with statistics: the correct way to summarize benchmark results. *Communications of the ACM*, 1986.

Ggerganov. Github - ggerganov/llama.cpp: Llm inference in c/c++. https://github.com/ggerganov/llama.cpp. Accessed: 2024-10-31.

Goyal, P. Accurate, large minibatch sg d: training imagenet in 1 hour. *arXiv preprint arXiv:1706.02677*, 2017.

Grand View Research. ARM-Based Servers Market Size, Share, & Trends Analysis Report By Core Type (ARM Cortex-A Core, ARM Cortex-M Core), By OS, By Processor, By Application, By Vertical, By Region, And Segment Forecasts, 2024 - 2030, 2024. URL https://www.grandviewresearch.com/industry-analysis/arm-based-servers-market-report. (n.d.).

Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y., et al. Deepseek-coder: When the large language model meets programming–the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.

Han, S., Liu, X., Kim, H., and Seo, J. A survey of processing-in-memory architecture and techniques. *Journal of Semiconductor Technology and Science*, 20 (6):513–526, 2020.

Hu, P., Liang, R., and Chen, K. Degpt: Optimizing decompiler output with llm. In *Proceedings 2024 Network and Distributed System Security Symposium (2024). https://api. semanticscholar. org/CorpusID*, volume 267622140, 2024.

Jouppi, N. P., Young, C., Patil, N., Patterson, D., Agrawal, G., Bajwa, R., and ... Yoon, D. H. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pp. 1–12, 2017.

Kalra, D. S. and Barkeshli, M. Why warmup the learning rate? underlying mechanisms and improvements. *arXiv preprint arXiv:2406.09405*, 2024.

Kang, U. et al. Co-architecting controllers and dram to enhance dram process scaling. In *Proceedings of The Memory Forum*, pp. 1–6, 2013.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Lachaux, M.-A., Roziere, B., Chanussot, L., and Lample, G. Unsupervised translation of programming languages. *arXiv preprint arXiv:2006.03511*, 2020.

Lamy-Poirier, J. Layered gradient accumulation and modular pipeline parallelism: fast and efficient training of large language models. *arXiv preprint arXiv:2106.02679*, 2021.

Langley, P. Crafting papers on machine learning. In *Proceedings of the 17th International Conference on Machine Learning (ICML 2000)*, 2000.

Lcvenshtcin, V. Binary coors capable or 'correcting deletions, insertions, and reversals. In *Soviet Physics-Doklady*, 1966.

Lee, C., Mahmoud, A., Kurek, M., Campanoni, S., Brooks, D., Chong, S., Wei, G.-Y., and Rush, A. M. Guess & sketch: Language model guided transpilation. In *The Twelfth International Conference on Learning Representations*, 2024.

Lewis, M. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461*, 2019.

Liu, A., Feng, B., Wang, B., Wang, B., Liu, B., Zhao, C., Dengr, C., Ruan, C., Dai, D., Guo, D., et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

Loshchilov, I. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

Markidis, S., Der Chien, S. W., Laure, E., Peng, I. B., and Vetter, J. S. Nvidia tensor core programmability, performance & precision. In *2018 IEEE international parallel and distributed processing symposium workshops (IPDPSW)*, pp. 522–531. IEEE, 2018.

Microsoft. Windows on ARM FAQ, 2024. URL https://learn.microsoft.com/en-us/windows/arm/faq. Accessed: 2024.

Morgan, T. P. Inside amazon's graviton3 arm server processor. 2022.

Myers, G. J., Sandler, C., and Badgett, T. *The art of software testing*. John Wiley & Sons, 2011.

OpenAI. Hello gpt4-o. https://openai.com/index/hello-gpt-4o/, 2024. Accessed: 2024-10-31.

Radcolor. Radcolor/ARM-linux-gnueabi: Bleeding edge GNU gcc toolchains (cc only) built from sources with latest binutils and glibc (for arm). https://github.com/radcolor/arm-linux-gnueabi, n.d. GitHub.

Rahman, T. N., Khan, N., and Zaman, Z. I. Redefining computing: Rise of arm from consumer to cloud for energy efficiency. *arXiv preprint arXiv:2402.02527*, 2024.

Roziere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Shazeer, N. and Stern, M. Adafactor: Adaptive learning rates with sublinear memory cost. In *International Conference on Machine Learning*, pp. 4596–4604. PMLR, 2018.

Shilov, A. Apple finishes dumping Intel entirely, touts results, 8 2023. URL https://www.tomshardware.com/news/apple-silicon-transition-complete-dumps-intel.

Sites, R. L., Chernoff, A., Kirk, M. B., Marks, M. P., and Robinson, S. G. Binary translation. *Communications of the ACM*, 36(2):69–81, 1993.

Tan, H., Luo, Q., Li, J., and Zhang, Y. Llm4decompile: Decompiling binary code with large language models. *arXiv*, 2024.

Vaswani, A. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

Wang, H., Qu, W., Katz, G., Zhu, W., Gao, Z., Qiu, H., Zhuge, J., and Zhang, C. Jtrans: Jump-aware transformer for binary code similarity detection. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022.

Wang, Y., Wang, W., Joty, S., and Hoi, S. C. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859*, 2021.

Wei, M.-t., Lin, Y.-S., and Lee, C.-R. Performance optimization for infiniband virtualization on qemu/kvm. In *2019 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, pp. 19–26. IEEE, 2019.

Woo, S. The rise of arm in data centers: Apple, amazon, and beyond. *IEEE Micro*, 40(5):10–15, 2020.

Xu, X., Feng, S., Ye, Y., Shen, G., Su, Z., Cheng, S., Tao, G., Shi, Q., Zhang, Z., and Zhang, X. Improving binary code similarity transformer models by semantics-driven instruction deemphasis. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023.

Yu, Z., Cao, R., Tang, Q., Nie, S., Huang, J., and Wu, S. Order matters: Semantic-aware neural networks for binary code similarity detection. In *Proceedings of the AAAI conference on artificial intelligence*, 2020.

Zhu, Q., Guo, D., Shao, Z., Yang, D., Wang, P., Xu, R., Wu, Y., Li, Y., Gao, H., Ma, S., et al. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. *arXiv preprint arXiv:2406.11931*, 2024.

A APPENDIX

A.1 Analysis of Assembly Code Variations

In this section, we present a detailed analysis of the variations between ground truth and predicted assembly code from our evaluation benchmark. Our analysis focuses on two key aspects: (1) functionally equivalent code with non-zero edit distance, and (2) incorrect implementations that fail the test cases. For functionally equivalent code, we identified several common patterns that maintain correctness despite syntactic differences. These patterns range from simple variations(Table 5) like register allocation and commutative operations, to complex patterns that combine multiple types of differences (Table 6).

Table 5. Simple Variation Patterns in Functionally Equivalent Code

Prog ID	Edit Dist	Example
P75	8	**Operands in arithmetic operations can be reordered if operation is commutative** Ground truth: `add r1, r2, r3` Predicted: `add r1, r3, r2`
P108	16	**Different registers can be chosen for temporary values while maintaining same data flow** Ground truth: `mov r2, r0; add r2, r2, #1` Predicted: `mov r3, r0; add r3, r3, #1`
P8	12	**Local variables can be stored at different stack locations while maintaining correct access patterns** Ground truth: `str r1, [fp, #-8]; str r2, [fp, #-12]` Predicted: `str r1, [fp, #-12]; str r2, [fp, #-8]`
P119	6	**Compiler-generated symbol names can differ while referring to same data** Ground truth: `.word out.4781` Predicted: `.word out.4280`
P135	12	**Multiple instructions can be combined into single equivalent instruction** Ground truth: `mov r3, r0;` `str r3, [fp, #-8]` Predicted: `str r0, [fp, #-8]`
P162	4	**Stack frame offsets can vary while maintaining correct variable access** Ground truth: `strb r3, [fp, #-21]` Predicted: `strb r3, [fp, #-17]`
P88	23	**Memory allocation sizes can vary if sufficient for program needs** Ground truth: `mov r0, #400` Predicted: `mov r0, #800`
P103	52	**Different instruction sequences can achieve same logical result** Ground truth: `cmp r3, #0; and r3, r3, #1; rsblt r3, r3, #0` Predicted: `rsbs r2, r3, #0; and r3, r3, #1; and r2, r2, #1; rsbpl r3, r2, #0`
P69	50	**Constants can be loaded directly or from literal pool** Ground truth: `mvn r3, #-2147483648` Predicted: `ldr r3, .L8; .L8: .word 2147483647`

Table 6. Complex Variation Patterns with Multiple Differences

Prog ID	Edit Dist	Combined Patterns and Examples

P128	78	**Multiple Optimization Patterns:**
		Groud truth: `mul r1, r2, r3`
		Predicted:
		`lsl r1, r2, #2;`
		`add r1, r1, r2`

P113	74	**Memory and Instruction Patterns:**
		Ground truth:
		`str r1, [fp, #-12]`
		`mov r3, r2`
		`add r3, r3, #4`
		Predicted:
		`str r1, [fp, #-8]`
		`add r2, r2, #4`

For incorrect implementations, we focused particularly on cases with small edit distances to understand how subtle differences can lead to functional failures (Table 7). These cases often involve critical errors in immediate values, register management, or memory access patterns that fundamentally alter the program's behavior.

Table 7. Analysis of Critical Errors with Small Edit Distances

Prog ID	Edit Dist	Example
P37	1	**Incorrect immediate value causes wrong division factor and early loop termination** Ground truth: `asr r2, r2, #2` Predicted: `asr r2, r2, #1`
P127	1	**Array index offset error causes wrong element comparison** Ground truth: `sub r3, r3, #2` Predicted: `sub r3, r3, #1`
P63	12	**Register overwrite corrupts loop counter before multiplication** Ground truth: `mov r0, r2; ldr r1, [r3, r1, lsl #2]; mul r0, r0, r1` Predicted: `ldr r0, [r3, r1, lsl #2]; mul r0, r0, r1`
P153	17	**Incorrect instruction sequence fails to compute absolute value** Ground truth: `sub r2, r2, r3; cmp r2, #0; rsblt r2, r2, #0` Predicted: `sub r1, r2, r3; eor r2, r1, r2; sub r2, r2, r1`
P47	19	**Mismatched memory access offsets cause incorrect data retrieval** Ground truth: `str r1, [fp, #-404]; ldr r2, [fp, #-404]` Predicted: `str r1, [fp, #-404]; ldr r2, [r3, #-20]`

x86

```
Complex Register Operations
movq %rdi, -16(%rbp)
movl %esi, -20(%rbp)
movss %xmm0, -24(%rbp)

Memory Addressing Operations
movq -16(%rbp), %rax
movslq -28(%rbp), %rcx
movss (%rax,%rcx,4), %xmm0

Comparison and Jump
cmpl -20(%rbp), %eax
jge .LBB0_10

Native FPU Support
subss (%rax,%rcx,4), %xmm0
cvtss2sd %xmm0, %xmm1
ucomisd %xmm1, %xmm0
```

ARMv5

```
32-bit Register Usage
ldr r3, [fp, #-12]
add r3, r3, #1

Memory Addressing Operations
lsl r3, r3, #2
add r3, r2, r3

Comparison and Jump
cmp r2, r3
blt .L7

External FPU Calls
bl __aeabi_fsub
bl __aeabi_fcmpgt
```

ARMv8

```
Mixed Width Registers
ldr w8, [sp, #4]
ldrsw x9, [sp]

Memory Addressing Operation
ldr s1, [x8, x9, lsl #2]

Comparison and Jump
subs w8, w8, w9
cset w8, ge
tbnz w8, #0, LBB0_10

Native FPU Support
fsub s0, s0, s1
fcmp d0, d1
```

Figure 6. Example Assembly Code Generated from Identical Source Program for x86, ARMv5, and ARMv8, with aligned assembly segments highlighted by functionality.

A.2 Assembly Code Comparison Across Architectures

Figure 6 provides sample assembly code segments across different ISAs, all for the same program. We showcase some interesting cases, including how ARM has major differences between v5 and v8, despite both being RISC architectures. As the ISA evolves, the binary compatibility changes—a program compiled for ARMv5 might not be compatible with ARMv8.

Further, we highlight distinct challenges for transpilers in understanding certain constructs, such as mixed-width register operands, comparison-and-jump definitions, and the varying number of instructions needed to achieve the same functional objective. These samples showcase how ISAs are truly unique "languages," and why we believe (and evaluate) the efficacy of an LLM in transpiling between them.

A.3 Alignment

Building upon the *Guess & Sketch* method proposed by Lee et al. (Lee et al., 2024) for cross-architecture transpilation using BART, we investigated the application of this neural machine translation model to convert x86 assembly code into ARM assembly code. However, BART's limitation of a 1024-token context window poses significant challenges when dealing with large assembly functions. This difficulty is further exacerbated by the substantial differences between x86 and ARM instruction sets, which necessitate the preservation of complex semantic relationships that often extend beyond the fixed window size.

To address these challenges, we attempted a semantic-aware code segmentation pipeline that facilitates effective translation while maintaining both local instruction patterns and the overall program structure across different architectures. Intuitively, we aimed to "elevate" the code up to the IR level to implement alignment.

The goal targeted precise code alignment within the limitations of BART's 1024-token context window. Our initial experiments with fixed-window segmentation resulted in a modest accuracy rate of 4.40%, highlighting the inadequacy of simple segmentation methods in preserving essential program semantics. By implementing our semantic-aware pipeline, we significantly improved the translation accuracy to 16.46%, effectively maintaining both local and global semantics during cross-architecture assembly translation. However, more work needs to be done, and we found that the larger context window size, as described in the main paper, effectively side-stepped this issue of alignment.